

Python Tip: `map()` and `filter()`

Want to apply a function to every item in a sequence or filter elements? Use `map()` and `filter()` for cleaner and more efficient code.

Using `map()`

The `map()` function applies a function to all items in a list. It returns a map object, which is an iterator that can be converted into a list.

```
def myfunc(x):  
    return x**2 + 1  
  
numbers = [1, 2, 3, 4, 5]  
  
new_numbers = map(myfunc, numbers)  
  
print(list(new_numbers))  # [2, 5, 10, 17, 26]
```

```
[2, 5, 10, 17, 26]
```

Using `filter()`

The `filter()` function filters elements from an iterable based on a condition. It returns an iterator with elements that satisfy the condition.

```
def is_even(x):  
    return x % 2 == 0  
  
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
  
even_numbers = filter(is_even, numbers)  
  
print(list(even_numbers))  # [2, 4, 6, 8, 10]
```

```
[2, 4, 6, 8, 10]
```

Why This is Better Than Loops

Both `map()` and `filter()` use lazy evaluation, meaning they only process elements when called downstream.

By contrast, a loop will process and store all elements immediately.

```
def myfunc(x):  
    return x**2 + 1  
  
numbers = [1, 2, 3, 4, 5]  
  
# This stores all results in memory immediately  
list1 = [myfunc(x) for x in numbers]  
  
# This only computes when needed  
list2 = map(myfunc, numbers)  
  
print(list(list2))
```

```
[2, 5, 10, 17, 26]
```


Wrap-Up

Now you can use `map()` and `filter()` to modify and filter sequences with ease.

Follow me for more tips.

Shep Bryan IV